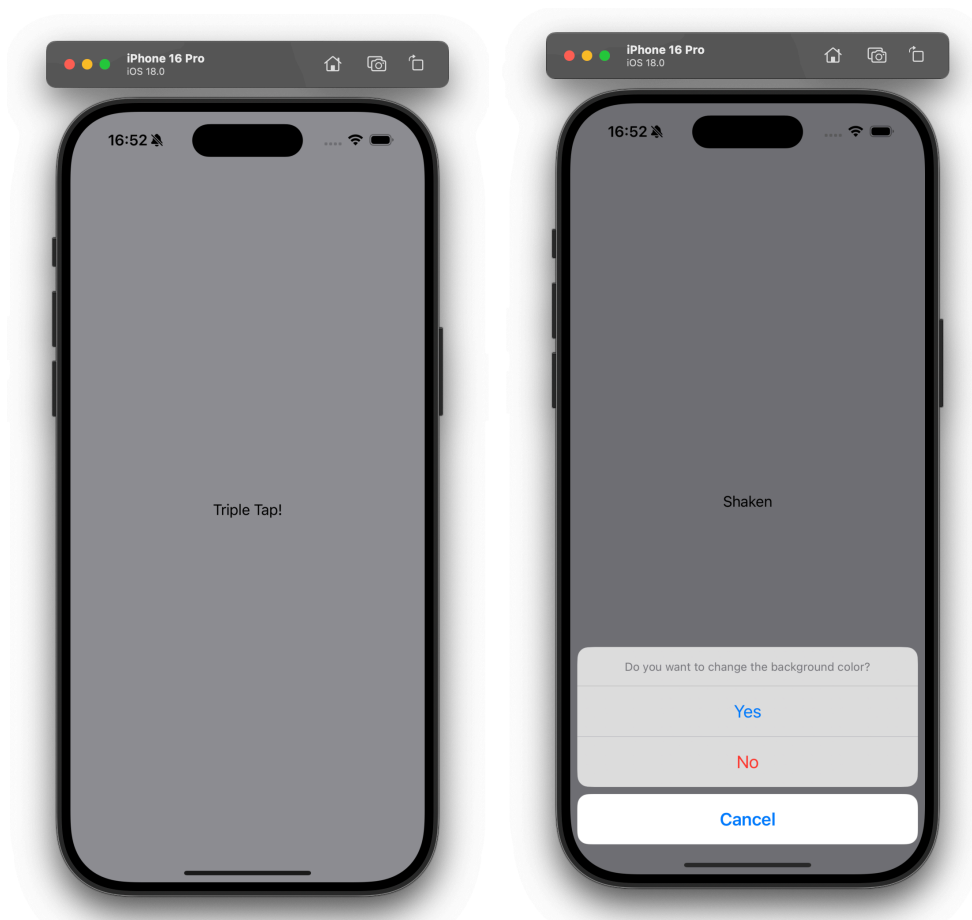


Zadanie 3 iOS — Wykorzystanie sprzętowych zasobów systemu mobilnego w SwiftUI

Należy stworzyć widoki spełniające poniższe wymagania:

1. Aplikacja powinna składać się z **dwóch widoków**, pomiędzy którymi można się dowolnie przełączać (np. za pomocą przycisku, gestu lub innego mechanizmu nawigacji).
2. **Pierwszy widok:**
 - (a) Widok powinien zawierać napis **"Listening to gestures..."** oraz domyślnie mieć tło w kolorze innym niż biały.
 - (b) Treść napisu powinna zmieniać się w zależności od wykonanego gestu: pojedyncze, podwójne i potrójne tapnięcie, długie przytrzymanie (long press), pinch oraz drag/swipe.
 - (c) Potrząśnięcie urządzeniem powinno wywołać okno dialogowe z przyciskami **Yes/No/Cancel** i tytułem **"Do you want to change the background color?"**.
 - i. Wybranie opcji **Yes** zmienia tło widoku na losowy kolor.
 - ii. Kolor tła powinien być także zmieniany na drugim widoku i powinien być losowy, inny niż na pierwszym widoku (choć oczywiście przez losowanie może w rzadkich przypadkach pojawić się kolor podobny lub ten sam). Jeśli kolor nigdy nie był zmieniany, drugi widok powinien pozostać biały.
 - iii. Pozostałe przyciski zamykają dialog bez zmian.
3. **Drugi widok:**
 - (a) Domyślnie posiada białe tło. Potrząśnięcie urządzeniem powinno zmienić kolor tła na losowy (bez wyświetlania dialogu).
 - (b) Po kliknięciu przycisku **"Get localization"** aplikacja powinna wyświetlić szerokość i długość geograficzną oraz adres.



Rys. 1 — Wykrywanie gestów (po lewej) oraz potrząśnięcia (po prawej) na pierwszym widoku



Rys. 2 — Zmiana koloru tła (po lewej) oraz wyświetlanie danych lokalizacyjnych (po prawej) na drugim widoku

Instrukcja do zadania 3

Część I – Wykrywanie gestów

1. Utwórz nowy projekt w SwiftUI i zmodyfikuj zawartość domyślnego widoku tak, aby zawierał kontrolkę Text zbindowaną (związaną) z polem message:

```
1 import SwiftUI
2
3 struct ContentView: View {
4     @State private var message = "Listening to gestures..."
5
6     var body: some View {
7         VStack {
8             Text(message)
9         }
10    }
11 }
```

2. Dodaj podstawowe gesty do kontrolki VStack. Wykrywanie kliknięć realizuje się za pomocą metody .onTapGesture, gdzie parametr count określa liczbę tapnięć wymaganych do wywołania zdarzenia. Przykład wykrywania pojedynczego kliknięcia:

```
1 .onTapGesture(count: 1) { self.message = "Tap!" }
```

3. Przetestujmy aplikację. Zauważmy, że gesty są rozpoznawane wyłącznie w obrębie tekstu, ponieważ wykrywanie gestów przypisane jest do konkretnej kontrolki. Aby łatwiej zobrazować problem, można dodać tło do VStack:

```
1 .background(.gray)
```

4. Aby umożliwić wykrywanie gestów na całym ekranie, należy rozszerzyć VStack na pełną powierzchnię ekranu, na przykład przy pomocy metody .frame(maxWidth: .infinity, maxHeight: .infinity).
5. Dodaj pozostałe gesty: długie przytrzymanie (metoda .onLongPressGesture) oraz gesty pinch i swipe, które można obsłużyć poprzez metodę .gesture(...) z użyciem struktur MagnifyGesture oraz DragGesture obsługując zdarzenia kolejno onChanged oraz onEnded.
6. Aby dodać reakcję na potrząśnięcie urządzeniem, zdefiniuj prywatną @State zmienną color, a następnie utwórz rozszerzenie dla NSNotification.Name:

```
1 extension NSNotification.Name {
2     public static let deviceDidShakeNotification = NSNotification.Name("
3         DeviceShakeNotification")
4 }
```

7. Zarejestruj nowe zdarzenie potrząśnięcia w rozszerzeniu klasy UIWindow, nadpisując metodę motionEnded:

```
1 open override func motionEnded(_ motion: UIEvent.EventSubtype, with event: UIEvent?) {
2     super.motionEnded(motion, with: event)
3     NotificationCenter.default.post(name: .deviceDidShakeNotification, object: event)
4 }
```

8. Odbierz notyfikację w głównym widoku:

```
1 .onReceive(NotificationCenter.default.publisher(for: .deviceDidShakeNotification)) {
2     _ in self.message = "Shaken"
3 }
```

9. Dodaj zmianę koloru tła po potrząśnięciu:

```
1 self.color = Color(
2     red: .random(in: 0...1), green: .random(in: 0...1), blue: .random(in: 0...1))
```

10. Aby potrząsnąć ekranem, należy wybrać z emulatora opcję **Device -> Shake** albo użyć skrótu **⌘ Z**. Tworzenie losowego koloru będzie używane w innych miejscach programu, więc dobrze jest stworzyć statyczną strukturę w której umieścimy metodę zwracającą losowy kolor. Utwórz pomocniczą strukturę **ColorHelper** i użyj jej w punkcie 9:

```
1 enum ColorHelper {  
2     static func getRandomColor() -> Color {  
3         Color(  
4             red:   .random(in: 0...1),  
5             green: .random(in: 0...1),  
6             blue:  .random(in: 0...1)  
7         )  
8     }  
9 }
```

11. Aby dodać okno dialogowe, wprowadź zmienną `@State private var isShowingDialog = false`, a następnie użyj komponentu `confirmationDialog`:

```
1 .confirmationDialog(  
2     "Do you want to change the background color?",  
3     isPresented: $isShowingDialog,  
4     titleVisibility: .visible  
5 ) {  
6     Button("Yes") { self.color = ColorHelper.getRandomColor() }  
7     Button("No", role: .destructive) { }  
8     Button("Cancel", role: .cancel) { }  
9 }
```

12. Po zakończeniu konfiguracji pierwszy widok powinien w pełni reagować na gesty oraz potrząśnięcia.
13. Następnie utwórz **drugi widok**. Przekaż do niego informację o tym, czy urządzenie było potrząśnięte. Można to zrealizować poprzez zmienną typu `Bool` przekazaną do konstruktora widoku.
14. Przejście między widokami można zrealizować np. za pomocą `NavigationLink`:

```
1 NavigationLink(destination: SecondView(wasShaken: $wasShaken)) {  
2     Text("Go to the second view")  
3         .padding(10)  
4         .cornerRadius(10)  
5 }
```

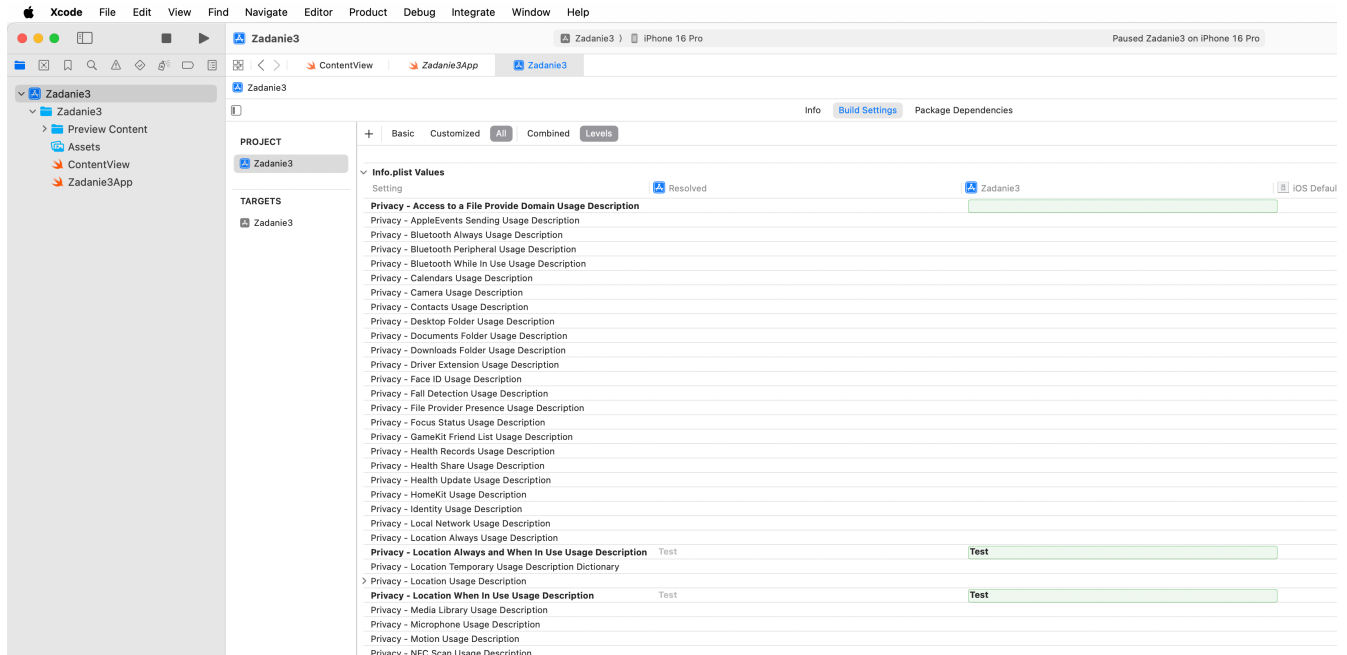
15. W drugim widoku ustaw tło w zależności od przekazanej wartości logicznej. Jeśli pierwszy widok był potrząśnięty, to kolor w drugim powinien być losowy (z wykorzystaniem `ColorHelper`) a jeśli nie był, to powinien pozostać biały. Podobnie jak wcześniej, dodaj obsługę potrząśnięcia urządzeniem.
16. Przetestuj gesty i przekazywanie parametrów do drugiego widoku. Na tym etapie wszystkie wymagania oprócz **3b)** powinny być spełnione.

Część II – Lokalizacja

1. Aby uzyskać dostęp do lokalizacji, dodaj odpowiednie wpisy w ustawieniach projektu (sekcja **Info**):

- **Privacy — Location Always and When In Use Usage Description**
- **Privacy — Location When In Use Usage Description**

Brak tych wpisów spowoduje, że aplikacja nie poprosi użytkownika o dostęp, a pobrana lokalizacja będzie miała wartość **nil**.



Rys. 3 — Ustawienia projektu, w których należy podać powód dostępu do lokalizacji

2. Utwórz klasę `LocationManager`, odpowiedzialną za obsługę lokalizacji:

```
1 final class LocationManager: NSObject, CLLocationManagerDelegate, ObservableObject {
2     @Published var location: CLLocation?
3     private var manager = CLLocationManager()
4
5     func checkLocationAuthorization() {
6         manager.delegate = self
7         manager.requestWhenInUseAuthorization()
8         manager.startUpdatingLocation()
9     }
10
11     func locationManager(_ manager: CLLocationManager, didUpdateLocations locations: [
12         CLLocation]) {
13         self.location = locations.first
14     }
15 }
```

3. Następnie należy ją użyć. Tworzymy zmienną która będzie przechowywać instancję klasy `LocationManager` a także zmienne typu `string` `latitude`, `longitude` i `address`, które będą używane w kontrolkach typu `Text` do wyświetlenia danych geolokalizacji. Obsłuż przycisk "Get localization".

4. Poniższy kod pobierze do zmiennej `location` geolokalizację:

```
1 locationManager.checkLocationAuthorization()
2 guard let location = locationManager.location else { return }
```

5. Aby zapisać współrzędne geograficzne należy użyć:

```
1 self.latitude = location.coordinate.latitude.description
2 self.longitude = location.coordinate.longitude.description
```

6. Do pobrania i wyświetlenia adresu należy użyć metody `reverseGeocodeLocation`:

```
1 CLGeocoder().reverseGeocodeLocation(location) { placemarks, _ in
2   guard let p = placemarks?.first else { return }
3   self.address = [
4     p.location?.description,
5     p.thoroughfare,
6     p.subAdministrativeArea,
7     p.isoCountryCode,
8     p.country
9   ]
10  .compactMap { $0 }
11  .joined(separator: ", ")
12 }
```

7. Powyższy kod służy do zapisania w `self.address` wszystkich danych adresu, oddzielając je przecinkami. Należy go wywołać w akcji przycisku. Przetestuj aplikację. Po tej operacji wszystkie wymagania zadania powinny być spełnione.